

Finance & Procurement Analytics

Oracle EBS–Style Data Model | Snowflake + SQL + Power BI + Excel

Project Blueprint prepared for: Datapltr Data Analytics & BI Internship Drive

Candidate: Sanjay | B.E. CSE, Kumaraguru College of Technology (KCT), Coimbatore

Build Duration: 1 focused day (compressed sprint)

Stack: Snowflake (Medallion Architecture) → SQL (Advanced) → Power BI (Dashboarding + DAX) → Excel (companion analysis)

Why this project: Datapltr builds pre-built data models on top of enterprise applications like SAP, Oracle EBS, Workday, and Salesforce. This project simulates a simplified but realistic Oracle EBS Financials / Procurement data model — Accounts Payable (AP), Purchase Orders, and Vendor Management — and builds an end-to-end analytics pipeline on top of it, exactly the kind of work Datapltr does for clients.

1. Business Context — Why Finance & Procurement (Oracle EBS-style)

Oracle E-Business Suite (EBS) is one of the most widely deployed ERP systems for large enterprises, and its Financials and Procurement modules (Accounts Payable, Purchasing, General Ledger) generate some of the richest, most analytically demanding datasets in any organization. Companies that build analytics layers on top of Oracle EBS — which is exactly what Datapltr does for its clients — need engineers and analysts who understand:

- How transactional data (invoices, purchase orders, payments) flows and links together via foreign keys
- How to model this data into clean fact and dimension tables for reporting
- How to compute finance-specific KPIs (DSO, DPO, aging, spend concentration) that businesses actually track
- How to detect data quality issues (duplicate invoices, mismatched POs) that are common in real ERP extracts

Building a project on this exact domain — instead of a generic retail or eCommerce dataset — signals to interviewers that you understand enterprise data structures, not just SQL syntax. It gives you a natural, confident answer when they ask 'why did you choose this project?'

1.1 The Business Problem You Are Solving

You are simulating the role of a Data Analyst supporting a mid-size company's Finance & Procurement department. The Finance team currently pulls data manually from Oracle EBS extracts into spreadsheets to answer basic questions — which is slow, error-prone, and not scalable. Your project builds a proper cloud-based analytics pipeline (Snowflake) with a dashboard (Power BI) that answers these business questions on demand:

- Which vendors are we spending the most with, and is that spend too concentrated (supplier risk)?
- How long does it take us to pay our invoices (DPO), and are we paying late (penalty risk)?
- How much of our payables are overdue right now, broken into aging buckets (30/60/90+ days)?
- Are there duplicate or suspicious invoices in the system (fraud/error risk)?
- Are purchase orders and invoices matching correctly (3-way match: PO → Goods Receipt → Invoice)?

2. Data Architecture — Snowflake Medallion Model

You will build a classic **Bronze** → **Silver** → **Gold** medallion architecture in Snowflake. This is the same layered approach used in real enterprise data platforms (including the kind Dataplatform builds), and mentioning it by name in an interview immediately signals maturity beyond a basic SQL script.

Layer	Purpose	What You Do
Bronze (Raw)	Landing zone for raw extracts, unmodified	Load CSVs (Invoices, POs, Vendors, Payments, etc.)
Silver (Cleaned)	Cleaned, validated, standardized	Remove nulls/duplicates, standardize date formats & units
Gold (Business-Ready)	Star-schema fact & dimension tables	Build FACT_INVOICE, FACT_PAYMENT, DIM_VENDOR, etc.

Why this matters for complexity: a flat single-table SQL exercise looks like a classroom assignment. A layered pipeline with a proper star schema (fact + dimension tables) looks like production-grade data engineering — which is what will differentiate you in the SQL/technical round.

2.1 Star Schema Design (Gold Layer)

Table	Type	Key Columns
FACT_INVOICE	Fact	invoice_id (PK), vendor_id (FK), po_id (FK), invoice_date, due_date, amount, currency
FACT_PAYMENT	Fact	payment_id (PK), invoice_id (FK), payment_date, amount_paid, payment_method
FACT_PURCHASE_ORDER	Fact	po_id (PK), vendor_id (FK), po_date, po_amount, goods_receipt_date, po_status
DIM_VENDOR	Dimension (SCD Type 2)	vendor_id, vendor_name, category, region, payment_terms, effective_start, effective_end
DIM_DATE	Dimension	date_key, day, month, quarter, year, fiscal_period
DIM_GL_ACCOUNT	Dimension	account_id, account_name, cost_center, department

Note on DIM_VENDOR: making this an SCD Type 2 dimension (tracking vendor payment-terms or category changes over time with effective_start_date / effective_end_date / is_current) is the single most advanced modeling concept you can showcase in one day. It directly mirrors how Oracle EBS and other ERP systems handle master data changes, and almost no fresher-level portfolio project includes it.

3. Dataset Design — Tables & Synthetic Data

Since you won't have access to real Oracle EBS data, you'll generate a realistic synthetic dataset using Python (Faker library) or structured Excel formulas. Aim for a size that's big enough to be meaningful but fast to generate in a day.

Table	Recommended Row Count	Notes
Vendors	40–60	Mix of categories: Raw Materials, IT Services, Logistics, Office Supplies, Consulting
Purchase Orders	300–500	Linked to vendors; some POs deliberately left unmatched to invoices (for exception handling)
Invoices	600–900	Some invoices with no matching PO (non-PO invoices), some duplicates (intentional)
Payments	500–800	Some invoices intentionally left unpaid/overdue to populate aging buckets
GL Accounts	15–25	Cost centers / departments for expense categorization

3.1 Deliberately Inject Data Quality Issues

This is what makes the project 'complex' rather than a clean toy dataset. Real ERP extracts are messy — so intentionally build in these issues, then write SQL to detect and report them:

- **Duplicate invoices:** same vendor, same amount, invoice dates within 3 days of each other
- **Orphan invoices:** invoices with no matching purchase order (non-PO spend — a real audit red flag)
- **Price mismatches:** invoice amount differs from PO amount by more than 5% (3-way match failure)
- **Late payments:** payment_date well past due_date, to populate meaningful aging buckets
- **Missing/null fields:** a few missing payment_terms or vendor_region values, to test your Silver-layer cleaning logic

4. One-Day Build Plan — Hour by Hour

Time Block	Phase	What You Do
Hr 1	Data Generation	Write a Python (Faker) script or structured Excel sheets to generate the 5 tables above, including
Hr 2	Snowflake Bronze	Create a Snowflake account/warehouse, create BRONZE schema, load raw CSVs via Snowsight
Hr 3	Snowflake Silver	Create SILVER schema. Write SQL to clean nulls, standardize date formats, cast data types, de
Hr 4	Snowflake Gold — Star Schema	Build FACT_INVOICE, FACT_PAYMENT, FACT_PURCHASE_ORDER, and dimension tables. I
Hr 5	Core SQL Analysis	Write the 6 analytical queries detailed in Section 5 (DSO/DPO, aging buckets, vendor concentrat
Hr 6	Power BI — Data Model	Connect Power BI to Snowflake (or import Gold-layer CSVs), build relationships between fact/dim
Hr 7	Power BI — Dashboard Pages	Build 2 dashboard pages: (1) AP Aging & Cash Flow Overview, (2) Vendor Spend & Risk Analys
Hr 8	Excel Companion + Docs	Build a lightweight Excel budget-vs-actual variance sheet using pivot tables/formulas. Write REA

This is an aggressive but achievable single-day plan if you use synthetic data generation scripts rather than manual entry, and keep the Power BI design simple but polished rather than elaborate.

5. Core SQL Queries — In-Depth Logic

These six queries are your technical centerpiece. Each one demonstrates a different advanced SQL concept that the Datapltr JD explicitly lists as required or preferred (CTEs, window functions, subqueries, CASE, joins). Understand the *logic* behind each, not just the syntax — you may be asked to explain your reasoning in the SQL round.

5.1 Days Payable Outstanding (DPO) — CTE + Aggregate

Business question: On average, how many days does it take us to pay an invoice after it's issued? A rising DPO can mean cash-flow strain or vendor relationship risk.

```
WITH paid_invoices AS (  
  SELECT f.invoice_id, f.vendor_id, f.invoice_date, p.payment_date,  
         DATEDIFF('day', f.invoice_date, p.payment_date) AS days_to_pay  
  FROM FACT_INVOICE f  
  JOIN FACT_PAYMENT p ON f.invoice_id = p.invoice_id  
)  
SELECT vendor_id, ROUND(AVG(days_to_pay), 1) AS avg_dpo  
FROM paid_invoices  
GROUP BY vendor_id  
ORDER BY avg_dpo DESC;
```

5.2 Invoice Aging Buckets — CASE + Conditional Aggregation

Business question: How much of our unpaid payables is overdue, broken into standard aging buckets (0-30 / 31-60 / 61-90 / 90+ days)? This is the single most common finance dashboard metric.

```
SELECT  
  invoice_id, vendor_id, amount,  
  DATEDIFF('day', due_date, CURRENT_DATE) AS days_overdue,  
  CASE  
    WHEN DATEDIFF('day', due_date, CURRENT_DATE) <= 0 THEN 'Not Due'  
    WHEN DATEDIFF('day', due_date, CURRENT_DATE) BETWEEN 1 AND 30 THEN '0-30 Days'  
    WHEN DATEDIFF('day', due_date, CURRENT_DATE) BETWEEN 31 AND 60 THEN '31-60 Days'  
    WHEN DATEDIFF('day', due_date, CURRENT_DATE) BETWEEN 61 AND 90 THEN '61-90 Days'  
    ELSE '90+ Days'  
  END AS aging_bucket  
FROM FACT_INVOICE  
WHERE status = 'Unpaid';
```

5.3 Vendor Spend Concentration — Window Functions

Business question: Are we over-reliant on a small number of vendors (supplier concentration risk)? Uses window functions to compute each vendor's share of total spend and a running cumulative percentage (the basis of a Pareto / 80-20 analysis).

```
SELECT  
  vendor_id,  
  SUM(amount) AS total_spend,  
  RANK() OVER (ORDER BY SUM(amount) DESC) AS spend_rank,  
  ROUND(100.0 * SUM(amount) / SUM(SUM(amount)) OVER (), 2) AS pct_of_total,  
  ROUND(100.0 * SUM(SUM(amount)) OVER (ORDER BY SUM(amount) DESC)  
        / SUM(SUM(amount)) OVER (), 2) AS cumulative_pct  
FROM FACT_INVOICE  
GROUP BY vendor_id  
ORDER BY total_spend DESC;
```

5.4 Duplicate Invoice Detection — Self-Join

Business question: Flag potential duplicate invoices — same vendor, same amount, invoice dates within a 3-day window. This is a real fraud/error-control check finance teams run regularly.

```
SELECT a.invoice_id AS invoice_1, b.invoice_id AS invoice_2,
       a.vendor_id, a.amount, a.invoice_date, b.invoice_date
FROM FACT_INVOICE a
JOIN FACT_INVOICE b
  ON a.vendor_id = b.vendor_id
 AND a.amount = b.amount
 AND a.invoice_id < b.invoice_id
 AND ABS(DATEDIFF('day', a.invoice_date, b.invoice_date)) <= 3;
```

5.5 Three-Way Match Variance — Subquery + CASE

Business question: Does the invoiced amount match the original purchase order amount within an acceptable tolerance? Flags exceptions where invoice amount deviates more than 5% from the PO amount.

```
SELECT
  f.invoice_id, f.po_id, f.amount AS invoice_amount, po.po_amount,
  ROUND(100.0 * (f.amount - po.po_amount) / NULLIF(po.po_amount, 0), 2) AS pct_variance,
  CASE
    WHEN ABS(f.amount - po.po_amount) / NULLIF(po.po_amount, 0) > 0.05 THEN 'Exception -
  Review'
    ELSE 'Matched'
  END AS match_status
FROM FACT_INVOICE f
JOIN FACT_PURCHASE_ORDER po ON f.po_id = po.po_id;
```

5.6 Vendor History Snapshot — SCD Type 2 Query

Business question: What payment terms did a given vendor have at the time a specific invoice was issued (not today's terms)? This is only answerable because DIM_VENDOR is modeled as SCD Type 2 — a flat/current-only vendor table cannot answer this correctly.

```
SELECT f.invoice_id, f.invoice_date, v.vendor_name, v.payment_terms
FROM FACT_INVOICE f
JOIN DIM_VENDOR v
  ON f.vendor_id = v.vendor_id
 AND f.invoice_date BETWEEN v.effective_start_date AND v.effective_end_date;
```

Interview talking point: explain that this query pattern is exactly how historically-accurate reporting works in real ERP-backed data warehouses — it's the difference between 'what is true now' and 'what was true then', which matters enormously for audit and compliance in finance data.

6. Power BI Dashboard Design

6.1 Page 1 — AP Aging & Cash Flow Overview

- KPI cards: Total Outstanding Payables, Total Overdue Amount, Average DPO
- Stacked bar chart: Outstanding amount by aging bucket (0-30 / 31-60 / 61-90 / 90+)
- Line chart: Monthly payment trend (invoiced vs paid) over the last 12 months
- Table: Top 10 overdue invoices, sortable by days overdue
- Slicers: Date range, Vendor category, Region

6.2 Page 2 — Vendor Spend & Risk Analysis

- Pareto chart: Vendor spend concentration (bar + cumulative % line) — direct visual of the window-function query
- Donut chart: Spend by vendor category (Raw Materials, IT Services, Logistics, etc.)
- Matrix/table: 3-way match exceptions — invoices flagged for PO price variance, with drill-through to invoice detail
- Card: Number of duplicate invoices flagged this period

6.3 Suggested DAX Measures

Measure	DAX Logic (concept)
Total Outstanding	SUM of invoice amount where status = 'Unpaid'
Avg DPO	AVERAGEX over paid invoices of (payment_date - invoice_date)
% Overdue	DIVIDE(Overdue Amount, Total Outstanding) — using DIVIDE to safely handle zero-division
MoM Payment Trend	Uses CALCULATE + DATEADD/SAMEPERIODLASTYEAR against the date dimension for month-over-month

7. Excel Companion — Budget vs Actual Variance Sheet

A lightweight Excel artifact strengthens the 'Good to Have: Excel' line in the JD and shows you can work outside a BI tool when needed — many finance teams still live in Excel day-to-day.

- Pivot table: Actual spend by GL account/cost center vs a budgeted figure you define
- Variance column with conditional formatting (red/green) for over/under budget
- SUMIFS/XLOOKUP formulas pulling from the Gold-layer export rather than manual entry
- One simple chart: Budget vs Actual by department

8. Interview Talking Points — How to Present This

- **Opening line:** "I built a simplified Oracle EBS-style Finance & Procurement analytics pipeline, since that's one of the enterprise systems Datapltr builds data models for."
- **On architecture:** explain the Bronze/Silver/Gold medallion layers and why each exists (raw fidelity → data quality → business-ready star schema).
- **On SCD Type 2:** this is your strongest differentiator — be ready to explain what problem it solves (historically accurate reporting) and how `effective_start_date/effective_end_date` work.
- **On SQL:** be ready to explain the logic (not just recite) behind the aging-bucket CASE statement, the window-function Pareto query, and the duplicate-detection self-join.
- **On business impact:** frame every query in terms of a business outcome — cash flow visibility, fraud/error control, supplier risk — not just as a technical exercise.

9. Deliverables Checklist

- GitHub repo with README (architecture diagram + screenshots + key SQL snippets)
- 5 synthetic CSV source files (Bronze layer)
- Silver-layer cleaning SQL scripts
- Gold-layer star schema DDL + SCD Type 2 MERGE logic for `DIM_VENDOR`
- 6 analytical SQL queries with brief write-ups of the business question each answers
- Power BI .pbix file — 2 pages, slicers, 4-5 DAX measures
- Excel budget-vs-actual companion workbook